Cutting p99 Latency by 45% Under Peak Load

Profiling, TLS Tuning, Worker Pools, Caching and Connection Pooling

A simple-language guide to the project: summary, components, workflow and architecture

> **The project in one sentence (as written)**
> Improved p99 latency 45% under peak load by profiling hot paths in the request inspection path, tuning TLS handshakes and connection reuse, right-sizing async worker pools, and adding multi-tier caching and connection pooling.

Contents

1. Overall summary (in simple terms)
2. Decoding the project sentence, phrase by phrase
3. Key ideas explained simply (a mini-glossary)
4. Architecture diagram and a full walk-through of it
5. Components explained in detail
6. Workflows explained end to end
7. Where the 45% came from
8. Trade-offs, risks and safeguards
9. Quick recap

Note: the project sentence is a short summary. This guide describes a realistic, typical design for such a project so that every part of the sentence can be explained concretely. Component names and all millisecond figures are illustrative examples chosen for this guide, not measurements from a specific system.

1. Overall Summary (in Simple Terms)

Picture an **airport security checkpoint at rush hour**. Most travellers get through in a few minutes, but a small unlucky group gets stuck for a very long time: their ID takes ages to verify, they queue behind a desk with too few staff, or the officer has to phone head office to check their details. In computing, the checkpoint is a **proxy or gateway**: a server that every request passes through, where each request is decrypted, inspected for security, and forwarded to the real application.

p99 latency is the waiting time of that unlucky group. If you line up 100 requests from fastest to slowest, p99 is the time taken by request number 99. Only 1 in 100 requests is slower. It matters because busy users make many requests, so almost everyone eventually hits the slow ones, and because slow requests pile up during **peak load** (the busiest moments, such as a sale or the start of the working day).

The project made that slowest 1% about **45% faster at peak load** (for example, from about 400 milliseconds down to about 220). It did this with five practical improvements:

What was done	Airport analogy	What it fixed
Profiled the hot paths in request inspection	Filming the checkpoint to see exactly where people get stuck	Wasted CPU work, slow rules, lock waits and memory churn in the code every request runs
Tuned TLS handshakes and connection reuse	A fast lane for returning travellers who already showed their passport; keeping the door open instead of locking it after each person	Expensive encryption set-up repeated far too often
Right-sized async worker pools	Putting the right number of officers on each desk, and separating quick checks from slow paperwork	Requests waiting in queues behind slow tasks, or too many threads fighting over CPUs
Added multi-tier caching	A pocket notebook, a shared desk binder, and only then a phone call to head office	Repeated slow look-ups of the same login tokens and security policies
Added connection pooling	Shuttle buses that stay running between the checkpoint and the gates	Opening a brand-new network connection to backends and services for every request

> **In one breath**
> Measure the slowest 1% at peak → find where time is lost → reuse expensive encryption and connections → give the work the right number of workers → remember answers in fast caches → keep connections to other services warm → measure again. Result: the worst-case waits shrank by 45%.

2. Decoding the Project Sentence, Phrase by Phrase

Phrase	What it means in simple words
Improved p99 latency	Reduced the response time of the slowest 1% of requests. Latency is the time between sending a request and getting the answer.
45%	The slow-tail time dropped by almost half. Example: 400 ms became 220 ms.
Under peak load	Measured at the busiest traffic level, when queues form and problems are worst, not on a quiet day.
Profiling	Using tools that record where a running program spends its time (CPU, waiting, locks, memory). It turns guesses into facts.
Hot paths	The pieces of code that run for every request or run extremely often. Saving a little there saves a lot overall.
Request inspection path	The security checks each request goes through: reading headers, verifying login tokens, applying firewall (WAF) rules and access policies.
Tuning TLS handshakes	Making the encryption set-up step cheaper: TLS 1.3, session resumption, faster certificate types, shared session tickets.
Connection reuse	Keeping a client's connection open (keep-alive, HTTP/2) so many requests share one encrypted connection instead of each paying set-up costs.
Right-sizing async worker pools	Choosing the correct number of worker threads for each type of work, separating CPU work from waiting work, and limiting queue lengths.
Multi-tier caching	Remembering answers at several levels: in the worker's own memory (fastest), in a shared cache such as Redis, and only then asking the original service.
Connection pooling	Keeping a set of ready, open connections to backends and services, and lending them to requests instead of opening new ones each time.

3. Key Ideas Explained Simply (a Mini-Glossary)

3.1 Latency and percentiles

Averages hide pain. If 99 requests take 50 ms and one takes 5 seconds, the average looks fine but one user had a terrible experience. Percentiles describe the spread instead:

- **p50 (median):** half of requests are faster than this. The "typical" experience.
- **p95:** 95 out of 100 requests are faster.
- **p99:** 99 out of 100 are faster. This is the **tail**, the experience of the unlucky few.
- **Why p99 matters:** a single web page can trigger dozens of requests. With 50 requests per page, the chance that at least one hits the p99 tail is about 40%. So the tail is what users actually feel.

3.2 Peak load and why queues explode

When a server is 50% busy, a new request rarely waits. At 90% busy, it often waits behind others; at 95–99%, waits grow very quickly. Think of a motorway: traffic flows fine until it is nearly full, then a small increase causes a jam. This is why p99 gets dramatically worse at peak, and why removing even small amounts of work per request can shrink peak queues a lot.

3.3 Profiling and flame graphs

- **CPU profiling:** samples the program thousands of times per second to see which functions are running. Tools: `perf`, eBPF-based profilers, language profilers.
- **Off-CPU profiling:** records where threads are waiting instead of working (locks, disk, network). Waiting is often the real cause of tail latency.
- **Flame graph:** a picture where each bar is a function and its width is how much time it uses. Wide bars are where to look first.
- **Allocation and lock profiling:** shows memory churn and threads queuing for the same lock.

3.4 TLS handshake

TLS is the encryption behind HTTPS. Before any data flows, the client and server do a **handshake**: they agree on encryption, the server proves its identity with a certificate, and both compute shared keys. A full handshake costs network round trips and noticeable CPU (public-key maths). A **resumed** handshake reuses keys from an earlier session and is much cheaper. TLS 1.3 also needs fewer round trips than TLS 1.2.

3.5 Connection reuse

- **Keep-alive:** the connection stays open after a response so the next request can use it immediately.
- **HTTP/2 multiplexing:** many requests travel side by side over one connection instead of opening several.

- **Idle timeout:** how long an unused connection stays open before closing. Too short wastes handshakes; too long wastes memory.

3.6 Async worker pools

A **worker pool** is a fixed team of threads that pick up tasks from a queue. **Async** code lets one thread handle many requests by switching to other work while one is waiting for the network. Problems appear when the pool is the wrong size: too few workers means long queues; too many means threads fight over CPUs and switch constantly. Another classic problem is a **blocking call** (slow I/O) running on a thread meant for quick work, which stalls everything behind it.

- **Little's Law:** workers needed ≈ requests per second × time each request holds a worker. Example: 2,000 req/s × 0.004 s = 8 busy workers on average, plus headroom for peaks.
- **Bounded queue:** a queue with a maximum length, so overload is handled deliberately (reject fast or shed load) instead of letting waits grow without limit.

3.7 Caching

- **Cache:** a fast store of answers you already looked up. **Hit** = answer found; **miss** = must ask the slower source.
- **Hit ratio:** the percentage of look-ups answered by the cache. 95% hits means only 1 in 20 look-ups goes to the slow source.
- **TTL (time to live):** how long a cached answer is trusted before it must be refreshed.
- **Invalidation:** removing an answer early because the source changed (for example a revoked token).
- **Cache stampede:** many requests miss at once and all hit the slow source together. Prevented by letting one request fetch while the others wait for it (single-flight).

3.8 Connection pooling

Opening a new connection to another server means a TCP handshake, often a TLS handshake, and slow start-up. A **connection pool** keeps a set of open connections ready. A request borrows one, uses it, and returns it. Like a taxi rank where cars are waiting, instead of calling a taxi from across town every time.

4. Architecture Diagram and Full Walk-Through

The diagram on the next page shows the system after the improvements. The left zone is the data plane every request travels through. The right zone holds the caches the inspection engine uses, the TLS session cache, and the measurement and tuning loop that drove every change. Numbered circles on arrows show the order of events.

How to read the diagram	Meaning
Blue zone (left)	The proxy data plane. Boxes 1 to 7 are the stages each request passes through.
Orange zone (right)	Caches (L1, L2, L3, in orange) and the observability and tuning parts (A to D, in purple).
Green boxes	Things outside the proxy: clients, engineers, backend services, and the outcome.
Dark blue arrows 1–8	The live request path, from client to backend and back.
Brown arrows C1–C3	Cache look-ups. Double-headed because the question goes down and the answer comes back up.
Purple arrow R	TLS session resumption: the TLS stage stores and reads session state.
Purple arrows O1–O4	The tuning loop: measure, profile, decide, apply. Dashed O4 means it is a configuration change, not per-request traffic.

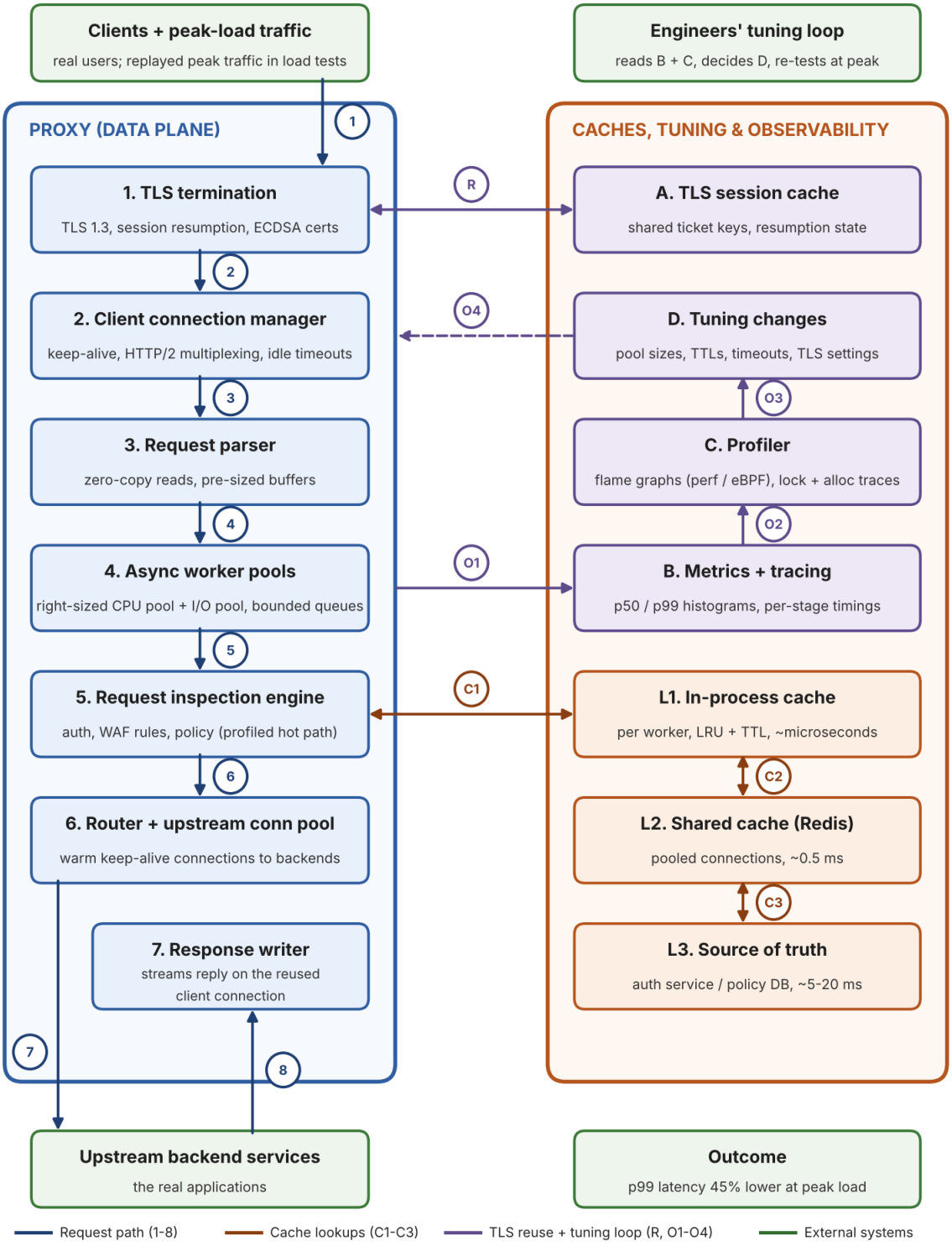


Figure 1. Latency-optimised proxy: request path, multi-tier cache and measurement-driven tuning loop

4.1 The zones

Blue zone – proxy (data plane). Every request passes through these seven stages in order. Each stage was a source of tail latency before the project, and each received a specific fix.

Orange zone – caches, tuning and observability. The orange boxes (L1, L2, L3) form the multi-tier cache used by the inspection engine. The purple boxes (A to D) are the TLS session cache plus the tools and decisions that made the improvements: metrics, profiler and the resulting configuration changes.

Green boxes – outside world. Clients (including load-test traffic that replays a peak), the engineers who run the tuning loop, the backend services, and the measured outcome.

4.2 Every box in the diagram

Box	Zone	One-line explanation
Clients + peak-load traffic	External	Real users, plus replayed peak traffic used to test changes safely.
Engineers' tuning loop	External	The people who read metrics and profiles, decide changes and re-test.
1. TLS termination	Data plane	Decrypts HTTPS. Tuned with TLS 1.3, session resumption and faster ECDSA certificates.
2. Client connection manager	Data plane	Keeps client connections open and shares them (keep-alive, HTTP/2).
3. Request parser	Data plane	Reads request bytes with minimal copying and pre-sized buffers.
4. Async worker pools	Data plane	Separate, right-sized pools for CPU work and waiting work, with bounded queues.
5. Request inspection engine	Data plane	Auth, WAF and policy checks. The profiled hot path; uses the caches.
6. Router + upstream conn pool	Data plane	Picks the backend and sends the request over a warm pooled connection.
7. Response writer	Data plane	Streams the backend's answer back over the same reused client connection.
A. TLS session cache	Support	Shared session-ticket keys and resumption state so returning clients skip full handshakes on any proxy node.
B. Metrics + tracing	Observability	Latency histograms (p50/p99) and per-stage timings for every request.

Box	Zone	One-line explanation
C. Profiler	Observability	Flame graphs, lock and allocation traces that show where time goes.
D. Tuning changes	Tuning	The resulting settings: pool sizes, cache TTLs, timeouts, TLS options.
L1. In-process cache	Cache	Tiny, per-worker memory cache. Answers in microseconds.
L2. Shared cache (Redis)	Cache	Shared by all proxy nodes, reached over pooled connections. About 0.5 ms.
L3. Source of truth	Cache	The real auth service or policy database. 5–20 ms; used only on misses.
Upstream backend services	External	The real applications that produce the response.
Outcome	Result	p99 latency 45% lower at peak load.

4.3 Every arrow in the diagram

Arrow	From → To	What travels along it
1	Clients → TLS termination	An encrypted request arrives (new connection or an existing one).
R	TLS termination ↔ TLS session cache	On a new connection, the TLS stage checks if the client can resume an old session; after a full handshake, it stores resumption data.
2	TLS → Client connection manager	Decrypted bytes belong to a connection that is kept alive for future requests.
3	Connection manager → Parser	A complete request is handed over for reading.
4	Parser → Worker pools	The parsed request becomes a task placed in the right pool's bounded queue.
5	Worker pools → Inspection engine	A worker picks up the task and runs the security checks.
C1	Inspection ↔ L1 cache	"Have I already verified this token / evaluated this policy?" Answer returns in microseconds on a hit.
C2	L1 ↔ L2 cache	On an L1 miss, ask the shared Redis cache; the answer is also copied back into L1.

Arrow	From → To	What travels along it
C3	L2 ↔ L3 source of truth	On an L2 miss, ask the real auth/policy service; the answer fills L2 and L1.
6	Inspection → Router + upstream pool	An allowed request is routed; a pooled connection to the backend is borrowed.
7	Router → Upstream backends	The request is sent over the warm connection.
8	Backends → Response writer	The response comes back and is streamed to the client on the same reused connection.
O1	Data plane → Metrics + tracing	Per-stage timings and latency histograms from every request.
O2	Metrics → Profiler	Slow stages identified in metrics tell engineers where to profile.
O3	Profiler → Tuning changes	Hotspots found in flame graphs turn into concrete changes.
O4	Tuning changes → Data plane	New settings and code changes are rolled out to the proxy (dashed: configuration, not traffic).

4.4 Why the diagram is shaped this way

- **The request path is a straight line down the left.** Each stage is a place where latency can hide; drawing them in order shows where each fix applies.
- **The cache sits beside the inspection engine (C1).** Caching was added exactly where the slow look-ups happened, not everywhere.
- **Cache tiers are stacked from fast to slow.** Most look-ups stop at L1; only a few travel all the way to L3.
- **The tuning loop is a closed circle (O1 → O2 → O3 → O4).** Every change was measured before and after, which is how the 45% figure was proven.

5. Components Explained in Detail

Each component below has a fact table (including the **problem found** and the **change made**) followed by a step-by-step description of how it works after the project.

5.1 External systems

Clients + peak-load traffic

What it is	Real users (browsers, mobile apps, partner systems) and synthetic traffic that replays a recorded peak hour.
Simple analogy	Real travellers on a holiday weekend, plus a full-scale fire drill that recreates the rush.
Why it matters	Tail latency only shows itself under realistic peak conditions. Replaying peak traffic let the team test every change safely before production.

How it works, step by step

1. Real clients connect over HTTPS and send requests; many send several requests per page or screen.
2. For testing, a load generator replays captured traffic shapes (same mix of endpoints, same connection behaviour, same burstiness) at peak and above-peak rates.
3. Both kinds of traffic enter the proxy through the same path (arrow 1), so measurements are comparable.

Engineers' tuning loop

What it is	The repeated process the team followed: measure, profile, change one thing, re-measure.
Simple analogy	A doctor who checks vital signs, runs a scan, prescribes one treatment, and checks again.
Rule followed	Change one thing at a time and keep it only if p99 improves at peak without hurting errors or cost.

How it works, step by step

1. Read p99 per stage from metrics (B) to find the slowest stage.
2. Profile that stage (C) to find the exact cause.
3. Design a fix and express it as a code change or setting (D).
4. Replay peak traffic and compare before and after.
5. Roll out gradually and confirm in production metrics.

5.2 Data plane components

1. TLS termination

What it is	The stage that decrypts incoming HTTPS traffic.
Simple analogy	The passport desk.
Problem found	Many clients performed a full handshake on every new connection. At peak, the CPU cost of public-key operations piled up, and each full handshake also cost extra network round trips. Resumption did not work across proxy nodes because each node had its own ticket keys.
Changes made	Enabled and preferred TLS 1.3 (one round trip). Enabled session resumption with session tickets whose keys are shared across all nodes and rotated regularly. Switched to ECDSA certificates (much cheaper to sign than large RSA keys) while keeping RSA as fallback for old clients. Enabled OCSP stapling so clients do not make a separate certificate-status check. Used hardware-accelerated ciphers (AES-GCM with CPU support).

How it works, step by step

1. A new connection arrives with a ClientHello message.
2. If the client presents a valid session ticket, the proxy decrypts it using the shared ticket key (arrow R) and resumes: no certificate signing, fewer round trips.
3. If not, a full TLS 1.3 handshake runs using the ECDSA certificate, and a new session ticket is issued so the next connection can resume.
4. Once keys are agreed, all data on the connection is decrypted with fast symmetric encryption.
5. The open, decrypted connection is passed to the connection manager (arrow 2).

2. Client connection manager

What it is	The part that keeps client connections open and decides when to close them.
Simple analogy	A door attendant who holds the door open for the next person instead of locking it every time.
Problem found	Idle timeouts were too short and some clients were forced onto HTTP/1.1 with one request per connection, so clients reconnected (and re-handshaked) constantly, especially at peak.
Changes made	Enabled HTTP/2 so one connection carries many parallel requests. Raised and tuned keep-alive and idle timeouts to match real client behaviour. Set sensible limits on concurrent streams per connection and total connections to protect memory.

How it works, step by step

1. Tracks every open client connection and its activity.
2. Reads requests from the connection; with HTTP/2, several requests can be in progress on one connection at once.
3. After a response, keeps the connection open for the next request instead of closing it.
4. Closes connections that stay idle longer than the tuned timeout, or when limits are reached (oldest/idle first).
5. Hands each complete request to the parser (arrow 3).

3. Request parser

What it is	Code that reads raw request bytes into a structured request (method, path, headers).
Simple analogy	Reading a form by pointing at the fields instead of copying the whole form out by hand.
Problem found	Profiling showed lots of small memory allocations and string copies per header, and buffers that kept growing and being re-allocated.
Changes made	Zero-copy parsing (headers point into the original buffer), pre-sized buffers reused from a pool, and lowercase header lookup tables built once instead of per request.

How it works, step by step

1. Borrows a buffer from a reusable buffer pool (no fresh allocation).
2. Scans the request line and headers in place, recording positions rather than copying text.
3. Applies size limits to reject absurdly large requests early.

4. Packages the result as a task and submits it to the right worker pool (arrow 4).

4. Async worker pools

What it is	The thread pools that run request work: an event-loop/CPU pool for quick computation and a separate I/O pool for anything that waits.
Simple analogy	Separate desks for quick stamp checks and for slow paperwork, each with the right number of officers.
Problem found	One big pool did everything. Some blocking calls (a synchronous look-up, a slow log write) ran on event-loop threads and froze every request queued behind them. The pool was also oversized, so threads fought for CPUs and spent time switching. Queues were unbounded, so at peak, wait times grew without limit.
Changes made	CPU pool sized to the number of cores (about one thread per core). Separate I/O pool for blocking work, sized with Little's Law. Bounded queues with fast rejection or load shedding when full. Moved all blocking calls off the event loop. Pinned hot threads to cores where helpful.

How it works, step by step

1. A parsed request arrives and is placed on the CPU pool's queue.
2. A free worker takes it immediately (with correct sizing, the queue is usually empty or very short).
3. If the work needs to wait (cache miss to L2/L3, network call), the worker starts the async operation and moves on to another request instead of sitting idle.
4. Any unavoidable blocking task is sent to the I/O pool so it cannot stall the CPU pool.
5. If a queue is full at extreme peak, the request is rejected quickly with a retry-able error instead of waiting seconds; this protects everyone else's latency.
6. The worker runs the inspection engine for the request (arrow 5).

5. Request inspection engine

What it is	The security checks every request must pass: authentication (verifying the login token), WAF rules (blocking attacks), and access policies.
Simple analogy	The security officer who checks your boarding pass, scans your bag and confirms you are allowed through.
Problem found	This was the main hot path. Flame graphs showed: token signature verification repeated for every request even for the same token; hundreds of regex rules evaluated one by one; regexes recompiled at runtime; a global lock around the policy table; and a network call to the auth service on every request.
Changes made	Cached token verification results and policy decisions (L1/L2/L3). Pre-compiled all regexes at load time and combined them into one multi-pattern matcher. Reordered rules so cheap, decisive checks run first. Replaced the global lock with a read-mostly, lock-free snapshot. Removed per-request logging string building on the hot path.

How it works, step by step

1. Extracts the token and the request attributes needed for checks.
2. Asks the L1 cache whether this token was recently verified and what the policy decision was (arrow C1).
3. On an L1 hit (most requests), uses the answer immediately. On a miss, the look-up flows to L2 and, if needed, L3 (arrows C2, C3), and the answer is saved on the way back.
4. Runs WAF rules using the pre-compiled combined matcher, cheapest rules first, stopping at the first block decision.
5. Reads policies from a lock-free snapshot so worker threads never wait for each other.
6. Allows, denies (with an error response) or forwards the request to the router (arrow 6).

6. Router + upstream connection pool

What it is	The stage that picks a backend server and sends the request to it using a pool of already-open connections.
Simple analogy	Shuttle buses that stay running between the checkpoint and the gates.
Problem found	New connections (TCP + TLS) were opened to backends for many requests. At peak this added handshake time and sometimes exhausted ports or hit connection limits on backends.
Changes made	Per-backend connection pools with a tuned minimum (kept warm) and maximum size, keep-alive, HTTP/2 to backends where supported, health checks, idle timeouts shorter than the backend's, and a short timeout when waiting for a free connection.

How it works, step by step

1. Matches the request to a backend cluster and picks a healthy server (load balancing).
2. Borrows an open connection from that server's pool (instant when one is free).
3. If none is free and the pool is below its maximum, opens a new one; if at maximum, waits briefly with a timeout.
4. Sends the request (arrow 7). Broken connections are discarded and replaced in the background.
5. Returns the connection to the pool after the response, ready for the next request.

7. Response writer

What it is	The stage that sends the backend's response back to the client.
Simple analogy	Handing the traveller their documents back through the same open door.
Changes made	Streams the response as it arrives instead of buffering it fully, and records final timings for metrics.

How it works, step by step

1. Receives the response from the backend (arrow 8).
2. Streams it to the client over the same kept-alive connection.
3. Records the end-to-end time and per-stage timings, which feed metrics (arrow O1).
4. Leaves the connection open for the client's next request.

5.3 Caches and support components

A. TLS session cache

What it is	Shared session-ticket encryption keys (and optionally a shared session store) used by every proxy node.
Simple analogy	A shared list of travellers already verified today, available at every passport desk.
Why shared	Load balancers can send a returning client to a different node. If keys are per-node, resumption fails and a full handshake is needed.
Safety	Keys are rotated frequently (for example hourly, keeping the previous key for decryption) to limit risk and keep forward secrecy strong.

How it works, step by step

1. A key-distribution job publishes the current and previous ticket keys to all nodes.
2. After a full handshake, the TLS stage issues a session ticket encrypted with the current key.
3. On reconnection to any node, the ticket is decrypted with the current or previous key and the session resumes.
4. Old keys are retired on schedule, so old tickets simply fall back to a full handshake.

B. Metrics + tracing

What it is	Latency histograms and distributed traces with timings for every stage of every request.
Simple analogy	Timestamps at every checkpoint on each traveller's boarding pass.
Key detail	Histograms (not averages) are needed to compute p99 correctly. Traces show which stage made a particular slow request slow.

How it works, step by step

1. Each stage records start and end times (arrow O1).
2. Times are added to histograms per stage and per endpoint, and a sample of requests is traced end to end.
3. Dashboards show p50, p95, p99 and p99.9 over time, especially during peaks.
4. Slow stages are flagged for profiling (arrow O2).

C. Profiler

What it is	Tools that record where the proxy spends time: CPU flame graphs, off-CPU (waiting) profiles, lock contention and memory allocation traces.
Simple analogy	A slow-motion camera on the checkpoint.
How used safely	Low-overhead sampling (for example eBPF-based) can run in production briefly during peak; detailed profiling runs during peak-replay load tests.

How it works, step by step

1. Captures CPU samples while peak traffic is flowing.
2. Builds flame graphs; wide bars point at expensive functions (for example regex matching or signature verification).
3. Captures off-CPU profiles to find threads waiting on locks or blocking I/O.
4. Captures allocation profiles to find memory churn (for example per-header string copies).
5. Turns findings into a ranked list of fixes (arrow O3).

D. Tuning changes

What it is	The concrete settings and code changes produced by the loop.
Simple analogy	The updated staff rota and procedures for the checkpoint.
Examples	Worker counts per pool; queue limits; cache sizes and TTLs; connection pool min/max and timeouts; keep-alive and idle timeouts; TLS versions, ciphers, certificate types and ticket-key rotation.

How it works, step by step

1. Each change is stored in configuration (or code) and reviewed.
2. It is tested with peak replay and compared against the previous p99.
3. It is rolled out gradually to production (arrow O4), watching p99, errors and CPU.
4. If p99 does not improve or anything regresses, it is rolled back.

L1. In-process cache

What it is	A small cache inside each proxy process (often per worker thread to avoid locks).
Simple analogy	A notebook in the officer's pocket.
What is stored	Token verification results (valid/invalid, user ID, expiry) and policy decisions for recent request keys.
Key settings	Size limit with LRU eviction (least recently used is removed first); short TTL (seconds) that never exceeds the token's own expiry.

How it works, step by step

1. Receives a look-up key (for example a hash of the token plus route) (arrow C1).
2. If present and not expired: returns the answer in microseconds. This is the most common case.
3. If missing: asks L2 (arrow C2). Only one request per key fetches; others wait for that result (single-flight).
4. Stores the returned answer for the next request.
5. Drops entries on TTL expiry or when told the source changed (invalidation).

L2. Shared cache (Redis)

What it is	A shared, in-memory cache cluster used by all proxy nodes.
Simple analogy	A binder on the shared desk that every officer can check.
Why needed	L1 is per process; without L2, every node and every worker would repeat the same slow L3 call. L2 makes a result found by one node usable by all.
Key settings	Reached over pooled connections (no per-request connect); short timeouts; longer TTL than L1; negative caching of invalid tokens for a short time.

How it works, step by step

1. Receives a look-up from L1 over a pooled connection (about 0.5 ms).
2. Hit: returns the answer; L1 stores it.
3. Miss: the request goes to L3 (arrow C3).
4. Stores L3's answer with a TTL so other nodes benefit.
5. If Redis is slow or down, the proxy times out quickly and goes straight to L3, so the cache can never make things slower than before.

L3. Source of truth

What it is	The real auth service or policy database that owns the data.
Simple analogy	Phoning head office.
Effect of caching	With high L1 and L2 hit ratios, L3 receives only a small fraction of the traffic it used to, so it is also less overloaded at peak, which makes its own responses faster.

How it works, step by step

1. Receives only cache misses, over pooled connections.
2. Verifies the token or evaluates the policy authoritatively.
3. Returns the answer, which fills L2 and L1 on the way back.
4. Publishes change events (for example token revoked, policy updated) so caches can invalidate early.

Upstream backends and outcome

Upstream backends	The applications themselves. They benefited from fewer new connections thanks to pooling.
Outcome	At peak load, p99 dropped by about 45%. p50 improved too, but less; the biggest gains were in the tail, where queuing and repeated handshakes and look-ups had been worst.

How it works, step by step

1. Backends receive requests over warm connections and reply.
2. Engineers compare peak p99 before and after each change and in total.
3. The final result is confirmed on real production peaks, not only in tests.

6. Workflows Explained End to End

6.1 Baseline measurement workflow

1. Choose the target metric: p99 end-to-end latency at peak traffic, plus p99 for each stage.
2. Make sure measurements use histograms and include every stage (TLS, queue wait, inspection, upstream, response).
3. Capture a real peak hour's traffic shape and build a replay test.
4. Run the replay at 100% and 120% of peak and record the baseline (for example p99 = 400 ms).
5. Break the baseline down by stage to see where the tail time goes (section 7 shows an example).

6.2 Profiling workflow (finding the hot paths)

1. Start peak replay so the system behaves as it does at the worst moment.
2. Run a CPU profiler and build flame graphs of the inspection path.
3. Run an off-CPU profiler to find time spent waiting (locks, blocking I/O, queues).
4. Run allocation profiling to find memory churn.
5. List each hotspot with its share of time and an estimated saving.
6. Fix the biggest one, re-run, and repeat. Profiles often change after each fix because the next bottleneck becomes visible.

Hotspot found (example)	Fix applied	Effect
Token signature verified on every request	Cache verification results in L1/L2	Large CPU and latency saving
Hundreds of regex rules run one by one; some recompiled at runtime	Pre-compile and combine into one multi-pattern matcher; cheap rules first	Inspection CPU time reduced
Global lock on the policy table	Lock-free, read-mostly snapshot	No more threads queuing for the lock at peak
Blocking call on event-loop threads	Moved to a separate I/O pool or made async	No more stalls of unrelated requests
Many small allocations per header	Zero-copy parsing and buffer pools	Less CPU and less garbage/allocator pressure
Full TLS handshakes dominate CPU at peak	TLS 1.3, shared tickets, ECDSA certs	Handshake CPU and round trips reduced

6.3 Live request workflow (after the improvements)

Step	Where	What happens, in simple words
1	Client → TLS	Request arrives. If the client already has an open connection, no handshake is needed at all.
R	TLS ↔ session cache	New connection: resume with a shared session ticket if possible; otherwise a full TLS 1.3 handshake with an ECDSA certificate.
2	Connection manager	The connection is kept alive; with HTTP/2 many requests share it.
3	Parser	Headers are read in place using a pooled buffer, with size limits.
4	Worker pools	The request is queued in the right-sized CPU pool; a worker picks it up almost immediately.
5	Inspection engine	Security checks start.
C1	L1 cache	Token and policy answer found in the worker's own memory (most requests stop here).
C2/C3	L2 / L3	Only on a miss: shared Redis, then the real auth/policy service, over pooled connections. Answers are cached on the way back.
5b	Inspection engine	WAF rules run via the combined matcher; policy read from a lock-free snapshot. Allow or deny.
6	Router + pool	A warm connection to a healthy backend is borrowed.
7	Backend	The request is processed by the application.
8	Response writer	The response streams back to the client on the same connection; timings go to metrics.

6.4 TLS handshake workflow: full versus resumed

Step	Full handshake (first visit)	Resumed handshake (returning client)
1	Client sends ClientHello with supported ciphers and a key share.	Client sends ClientHello plus its session ticket.
2	Proxy picks settings, sends its key share, certificate and a signature (expensive public-key work).	Proxy decrypts the ticket with the shared key (cheap) and skips the certificate signature.
3	Client verifies the certificate chain (OCSP status stapled, so no extra call).	Nothing to verify again.

Step	Full handshake (first visit)	Resumed handshake (returning client)
4	Both derive keys; encrypted data starts after one round trip (TLS 1.3).	Both derive keys; data starts after one round trip (or zero with 0-RTT, used only for safe requests).
5	Proxy issues a new session ticket for next time.	Proxy may issue a refreshed ticket.

Combined with connection reuse, most requests do not need a handshake at all, and most new connections use the cheap resumed path.

6.5 Multi-tier cache workflow

1. **Build the key:** the inspection engine hashes the token (never storing the raw token as a key) plus the route or policy context.
2. **L1 look-up:** in-process, per worker, microseconds. Hit → done.
3. **Single-flight on miss:** if several requests miss the same key at once, one fetches and the rest wait for its result, preventing a stampede.
4. **L2 look-up:** Redis over a pooled connection with a short timeout (for example 2 ms). Hit → copy to L1 → done.
5. **L3 look-up:** the auth/policy service over a pooled connection. Result is written to L2 and L1.
6. **TTL rules:** L1 TTL is short (seconds), L2 TTL longer (tens of seconds to minutes), and neither may exceed the token's expiry time.
7. **Negative caching:** invalid tokens are cached briefly as "invalid" so attackers cannot force repeated L3 calls.
8. **Invalidation:** when L3 publishes a change (token revoked, policy updated), L2 entries are deleted and L1 entries are dropped, so security decisions stay correct.
9. **Failure handling:** if L2 is slow or down, skip it after the timeout and go to L3; if L3 is down, follow the security policy (usually fail closed for auth).

6.6 Worker pool right-sizing workflow

1. Measure per-request CPU time and wait time at peak (from profiles and traces).
2. Size the CPU/event-loop pool to about the number of CPU cores, since more threads than cores only adds switching.
3. Size the I/O pool using Little's Law: threads ≈ peak rate of blocking tasks × average blocking time, plus headroom (for example 30%).
4. Find and remove every blocking call from event-loop threads (a profiler shows them as long stalls).
5. Give each pool a bounded queue; choose the limit from the maximum acceptable wait (for example queue length ≈ workers × acceptable wait ÷ task time).

6. Decide overload behaviour: reject quickly with a retry-able status, or shed low-priority traffic first.
7. Validate at 100% and 120% of peak: queue wait at p99 should stay small and flat.

6.7 Upstream connection pool workflow

1. At start-up, open a minimum number of connections to each backend (warm pool), so the first peak requests do not pay connection costs.
2. For each request, borrow an idle healthy connection; if none, open a new one up to the maximum; at the maximum, wait with a short timeout.
3. Use keep-alive (and HTTP/2 where supported) so one connection serves many requests.
4. Run background health checks; remove broken connections and replace them.
5. Close idle connections before the backend's own idle timeout, to avoid sending a request on a connection the backend just closed.
6. The same pooling pattern is used for connections to Redis (L2) and the auth/policy service (L3).

6.8 Rollout and verification workflow

1. Each change is first tested with peak replay against the baseline.
2. Deploy to a canary group of proxy nodes; compare their p99, error rate and CPU against the rest of the fleet.
3. Expand gradually (for example 10% → 50% → 100%), always during or across real peak periods.
4. Keep guardrails: automatic rollback if p99 or errors rise, cache hit ratio drops, or memory grows unexpectedly.
5. After all changes, compare full peak periods before and after: p99 down about 45%.

7. Where the 45% Came From

The table below is an **illustrative** p99 breakdown at peak, showing how each improvement contributed. Real p99 values of stages do not add up exactly, but this view explains the logic clearly.

Stage (at p99, peak)	Before	After	Main fix
TLS handshakes	90 ms	50 ms	TLS 1.3, shared session tickets, ECDSA, connection reuse
Queue wait in worker pools	110 ms	55 ms	Right-sized pools, blocking work moved off, bounded queues
Inspection CPU work	70 ms	50 ms	Profiling fixes: compiled regex, lock-free policy, zero-copy
Auth / policy look-ups	80 ms	35 ms	Multi-tier caching (L1, L2, L3)
Upstream connection set-up	30 ms	10 ms	Connection pooling and warm pools
Backend and other	20 ms	20 ms	Unchanged
Total (approximate)	400 ms	220 ms	About 45% lower

Why the tail improved more than the average

Tail latency is driven by queuing at peak and by occasional expensive events (a full handshake, a cache miss to a slow service, a new connection). Every change either removed one of those expensive events or reduced work per request, which lowers CPU usage, which in turn shrinks queues sharply near peak. That is why the p99 gain (45%) was larger than the median gain.

8. Trade-offs, Risks and Safeguards

Risk	Why it matters	Safeguard used
Stale cached security decisions	A revoked token could still be accepted from cache.	Short TTLs, TTL never beyond token expiry, invalidation events from L3.
Cache stampede	Many simultaneous misses overload L3.	Single-flight fetching, jittered TTLs, negative caching.
Cache memory growth	Large caches use RAM and can slow the process.	Size limits with LRU eviction; memory alerts.
Shared TLS ticket keys	A leaked key could decrypt resumed sessions.	Frequent rotation, secure key distribution, limited key lifetime.
0-RTT replay	Early data can be replayed by an attacker.	0-RTT only for safe, idempotent requests, or disabled.
Pool exhaustion	Requests wait for a free connection or worker.	Max sizes tuned from data, short acquire timeouts, metrics on wait time.
Load shedding rejects some requests	At extreme peak a few users see a retry-able error.	Only when queues are full; prioritise important traffic; clients retry with back-off.
Longer keep-alive uses memory	Many idle connections consume resources.	Tuned idle timeouts and connection caps.

9. Quick Recap

Question	Short answer
What was the problem?	The slowest 1% of requests through the proxy were very slow at peak traffic.
How was it found?	Peak-load replay, per-stage latency histograms, tracing, and profiling (CPU, off-CPU, locks, allocations).
What changed in TLS?	TLS 1.3, shared and rotated session tickets, ECDSA certificates, OCSP stapling, longer keep-alive and HTTP/2.
What changed in worker pools?	CPU pool sized to cores, separate I/O pool sized by Little's Law, blocking work moved off, bounded queues.
What caching was added?	L1 in-process, L2 shared Redis, L3 source of truth, with TTLs, single-flight, negative caching and invalidation.
What connection pooling was added?	Warm keep-alive pools to backends, Redis and the auth/policy service.
What was the result?	p99 latency about 45% lower at peak load, proven with before/after peak measurements.